

PROGRAMAÇÃO ORIENTADA A OBJETOS

Aula 13 – Padrão MVC em Java
Separação de responsabilidades em
aplicações orientadas a objetos

Análise e Desenvolvimento de Sistemas
Prof.: Cícero Santos

Agenda de Aula

- **Ao final da aula, o aluno deverá ser capaz de:**
 - Explicar o que é o padrão MVC;
 - Diferenciar Model, View e Controller;
 - Organizar um projeto Java em camadas;
 - Identificar erros comuns na separação de responsabilidades;
 - Relacionar MVC aos conceitos de encapsulamento, herança, polimorfismo e interface.

O que é MVC?

- **MVC é um padrão arquitetural que divide a aplicação em três partes:**
 - **Model**
 - Representa os dados e as regras de negócio.
 - **View**
 - Representa a interface com o usuário.
 - **Controller**
 - Recebe ações da View e coordena a comunicação com o Model.

Problema: interface muda com frequência

- **Interfaces de usuário mudam conforme a necessidade de cada usuário.**
 - **Exemplo: digitador**
 - prefere teclado;
 - telas simples;
 - agilidade na entrada de dados.
 - Exemplo: gerente
 - prefere relatórios;
 - gráficos;
 - informações resumidas.
- ** A regra de negócio pode ser a mesma, mesmo que a interface seja diferente.**

O problema do acoplamento

- **Imagine criar um sistema diferente para cada tipo de usuário.**
- **Se a interface estiver misturada com a lógica de negócio:**
 - qualquer alteração na tela pode quebrar a regra de negócio;
 - há repetição de código;
 - o sistema fica difícil de manter;
 - o custo de evolução aumenta.

**** O MVC reduz esse acoplamento.**

Componentes do MVC

- **Model**

- contém dados;
- contém regras de negócio;
- representa entidades do sistema.

- **View**

- xibe informações;
- recebe dados do usuário;
- não deve conter regra de negócio.

- **Controller**

- interpreta ações do usuário;
- chama operações do Model;
- decide qual resposta será exibida.

Regra principal do MVC

- **A lógica de negócio não deve depender da interface.**
- **O Model não deve saber:**
 - se a aplicação usa menu no terminal;
 - se usa interface gráfica;
 - se usa banco de dados;
 - se possui uma ou várias telas.

**** O Model deve representar o domínio do problema.**

Fluxo de interação



- **O usuário interage com a View.**
- **A View envia a ação ao Controller.**
- **O Controller interpreta a solicitação.**
- **O Controller chama o Model.**
- **O Model executa a regra de negócio.**
- **O Controller recebe o resultado.**
- **A View apresenta a resposta ao usuário.**

Por que usar?

- **O MVC permite separar:**

- lógica de negócio;
- lógica de apresentação;
- controle da interação;
- organização dos objetos.

>> continua

Por que usar?

• Vantagens:

- facilita manutenção;
- melhora legibilidade;
- reduz acoplamento;
- favorece reutilização;
- facilita testes;
- permite trocar a interface sem alterar a regra de negócio.

**** Reduzir ao máximo acoplamento e aumentar coesão na organização orientada a objetos.**

Aplicação MVC: CRUD

- **Vamos usar como exemplo um sistema simples de cadastro.**
- **CRUD significa:**
 - Create — cadastrar;
 - Read — consultar/listar;
 - Update — alterar;
 - Delete — excluir.

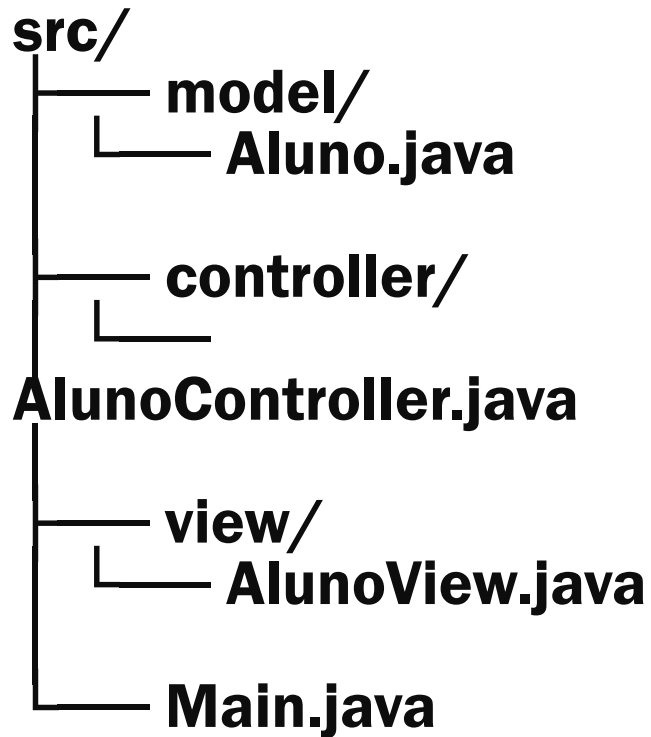
Exemplo da aula: Sistema de Cadastro de Alunos

Requisitos do CRUD de Alunos

- **O sistema deve permitir:**
 - cadastrar aluno;
 - buscar aluno por matrícula;
 - listar alunos;
 - alterar dados do aluno;
 - excluir aluno.

Essas funcionalidades serão organizadas em Model, View e Controller.

Organização dos pacotes



- Separar pacotes não é suficiente.
- Cada classe precisa ter responsabilidade correta.

Model: classe Aluno

- O Model representa os dados principais do sistema.

```
package model;

public class Aluno {
    private String nome;
    private String matricula;

    public Aluno(String nome, String matricula) {
        this.nome = nome;
        this.matricula = matricula;
    }

    public String getNome() {
        return nome;
    }

    public String getMatricula() {
        return matricula;
    }
}
```

Encapsulamento no Model

- O encapsulamento protege o estado interno do objeto.
- Os atributos não ficam públicos.
- O acesso ocorre por métodos.

```
private String nome;  
private String matricula;
```

```
public String getNome() {  
    return nome;  
}
```

// trabalha o uso de variáveis de instância private e métodos públicos como prática importante em classes Java .

Controller: controle das ações

- **O Controller coordena as ações da aplicação.**

```
package controller;

import model.Aluno;
import java.util.ArrayList;
import java.util.List;

public class AlunoController {
    private List<Aluno> alunos = new ArrayList<>();

    public void cadastrar(String nome, String matricula) {
        alunos.add(new Aluno(nome, matricula));
    }

    public List<Aluno> listar() {
        return alunos;
    }
}
```


View: interação com o usuário

- A View apresenta dados ao usuário.
- Ela não deve conter regra de negócio.

```
package view;

import model.Aluno;
import java.util.List;

public class AlunoView {
    public void mostrarAlunos(List<Aluno> alunos) {
        for (Aluno aluno : alunos) {
            System.out.println(aluno.getNome() + " - " +
aluno.getMatricula());
        }
    }
}
```

Main: início da aplicação

- A classe Main inicia e conecta as partes da aplicação.

```
import controller.AlunoController;
import view.AlunoView;

public class Main {
    public static void main(String[] args) {
        AlunoController controller = new AlunoController();
        AlunoView view = new AlunoView();

        controller.cadastrar("Ana", "2024001");
        controller.cadastrar("Carlos", "2024002");

        view.mostrarAlunos(controller.listar());
    }
}
```

MVC não é apenas criar pastas

- **Errado:**

- criar model, view e controller;
- colocar toda a lógica na View;
- deixar o Model imprimir mensagens;
- deixar o Controller fazer tudo sozinho.

- **Correto:**

- Model guarda dados e regras;
- View mostra informações;
- Controller coordena ações.

MVC e P00

- **O MVC organiza o projeto, mas os conceitos de P00 continuam essenciais.**
 - **Encapsulamento**
 - Atributos privados e acesso controlado.
 - **Herança**
 - Classes especializadas a partir de classes gerais.
 - **Polimorfismo**
 - Objetos diferentes tratados por uma referência comum.
 - **Interface**
 - Contrato comum implementado por classes diferentes.

Exemplo de herança no projeto

- Aluno é uma especialização de Pessoa.

```
public class Pessoa {
    private String nome;

    public Pessoa(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}

public class Aluno extends Pessoa {
    private String matricula;

    public Aluno(String nome, String matricula) {
        super(nome);
        this.matricula = matricula;
    }
}
```

Exemplo de interface

- A interface define um contrato.
- A classe decide como implementar esse contrato.

```
public interface Relatorio {  
    String gerarRelatorio();  
}  
  
public class Aluno extends Pessoa implements Relatorio {  
    private String matricula;  
  
    @Override  
    public String gerarRelatorio() {  
        return getNome() + " - " + matricula;  
    }  
}
```

Exemplo de polimorfismo

- **Objetos diferentes são tratados por uma mesma referência: Relatório.**

```
List<Relatorio> itens = new ArrayList<>();

itens.add(new Aluno("Ana", "2024001"));
itens.add(new Professor("Carlos", "P00"));

for (Relatorio item : itens) {
    System.out.println(item.gerarRelatorio());
}
```

Erros comuns no MVC

- **Evite:**
 - colocar `system.out.println` dentro do Model;
 - validar regra de negócio apenas na View;
 - deixar atributos públicos;
 - criar Controller sem função real;
 - misturar cadastro, exibição e regra em uma única classe;
 - usar MVC apenas como divisão de diretórios.

Obrigado.

Aula 13 – Padrão de Projeto

Análise e Desenvolvimento de Sistemas

Prof.: Cícero Santos

